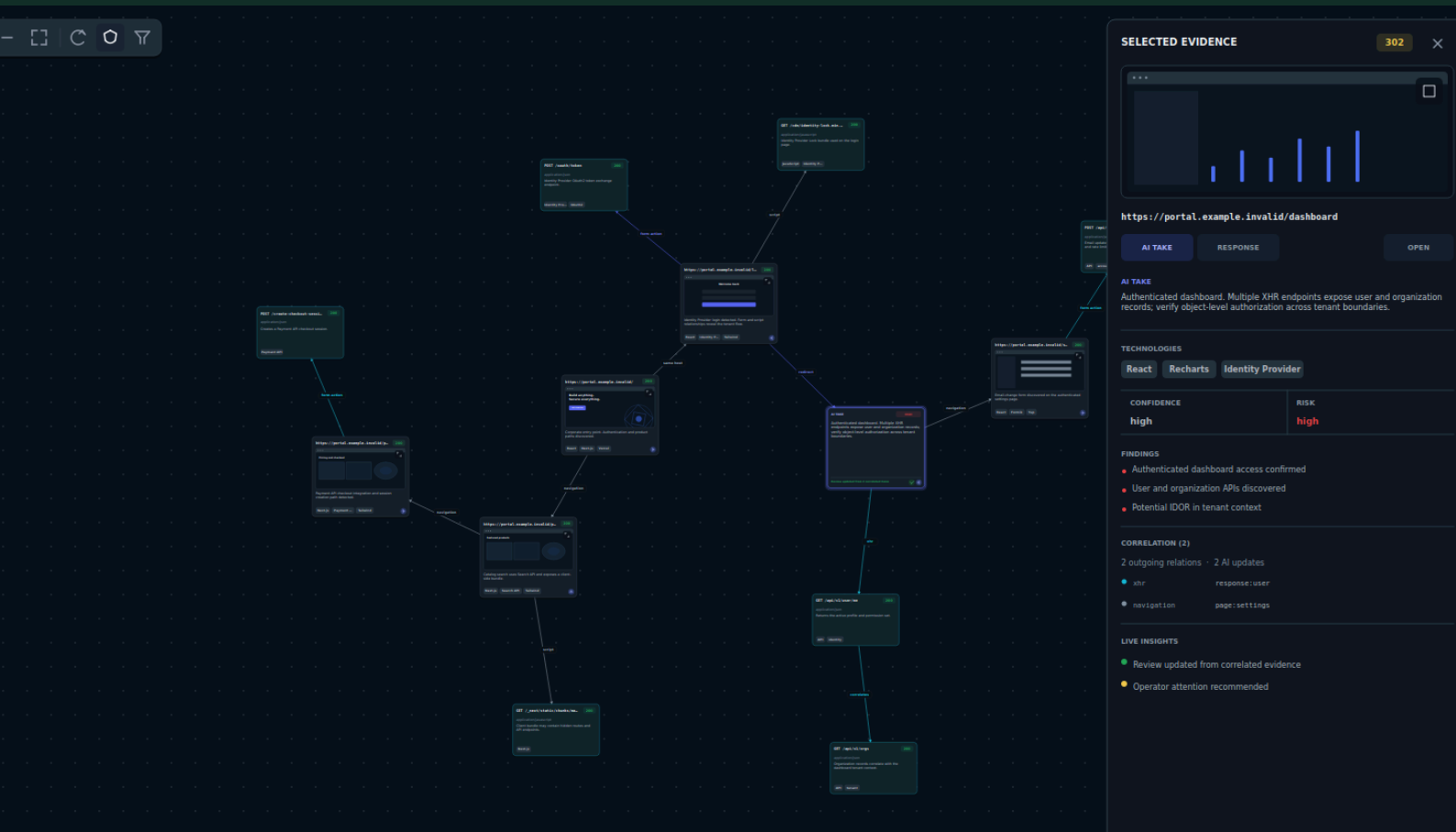


REDC2

Durable security orchestration with a native operator workspace.



WebRecon · Qt capture · synthetic fixture

Independently developed in Go and Qt, REDC2 combines persistent engagement state, evidence inspection, and component recovery for controlled security assessments. Development and environmental validation remain in progress.

Prepared for technical leadership at Cybears, Algeria.

Data and evidence atlas

The data layer defines the records that must remain consistent across execution, review, and recovery.

Engagement domain

A complete revision links evidence, identity, authority, actions, resources, and the work frontier.

Lifecycle store

Typed events preserve terminal outcomes, interruption causes, and attempt history.

Context model

Factual frames record identity, process placement, routes, scope, and validity.

Objective review

Collections and immutable groupings retain human-confirmed classification, source provenance, and score applicability.

PostgreSQL repository

Immutable generations and an atomic head establish the durable consistency boundary.

Specialist evidence store

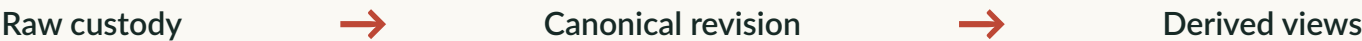
Content-addressed raw objects retain their digest, size, media metadata, and provenance.

Transformation engine

Versioned deterministic analysis records preserve inputs, results, deltas, and derivation history.

Derived projections and reporting

Compact snapshots serve Qt and Neo4j. A deterministic evidence index supports findings and report assembly.



Execution and analysis atlas

Application services evaluate work against canonical state. Execution providers perform bounded operations through declared interfaces.

Scheduler frontier

Capability evidence, resource health, context, and validated decisions determine eligibility and blockers.

Action coordinator

Prepared intent, execution receipts, and independent read-back observations preserve outcome custody.

Specialist adapters

HTTP/browser, packet/protocol, reverse/debug, artifact/mobile, and multi-environment producers share typed contracts.

Registered workers and process bundles

Immutable input envelopes, admitted artifacts, and live leases define execution boundaries.

Progress supervision

Separate heartbeat, progress, and overall deadlines distinguish liveness from useful work.

AI decision service

Revision-bound evidence, bounded provider work, and deterministic validation produce durable decision receipts.

Semantic experiments

Ordered plans retain checkpoints, mutation previews, exact gates, and cancellation causes.

Host/session attribution

Execution eligibility retains exact run and host identity; a channel on another host supplies no authority.

Evidence + context



Eligibility



Bounded execution

Runtime and engineering atlas

Runtime components manage availability and process custody. The engineering services repair or replace those components under explicit admission rules.

Composition kernel

Typed definitions, unpublished setup, generation leases, publication, and cleanup follow one ownership model.

Resource supervisors

Managed processes retain desired state, current health, backoff, and recovery ownership.

Agent/session components

Separate source components provide managed endpoint sessions. The stage0/helper work remains experimental.

Adaptive Capability Pipeline

A separate generation session builds a candidate for one scoped capability contract.

Specialist plugin registry

Bounded descriptors expose installed capabilities and explicit manual-required families.

Runtime switchboard

Provider identity, fencing, routing, and process custody form the permanent runtime anchor.

Watchdog and launcher

The Qt host embeds operator/tool windows; the first-entry launcher selects the workspace build.

Autonomic Engineering Plane

A persistent repair session owns diagnosis through validation, admission, replacement, and hand-back.

CI and deployment

Isolated checkouts reuse admitted caches; verified artifact sets pass compatibility checks before publication.

Incident



Owned repair



Verified hand-back

Operator and research atlas

The native workspace exposes the state produced by the underlying services. Research subsystems consume evidence through separate diagnostic interfaces.

Go/Qt bridge and operator shell

Versioned native calls carry compact state, lazy detail, and attributed decisions.

Network Graph

Observed hosts, sessions, routes, and relationships form a navigable topology.

Investigation Graph

Hypotheses and conclusions retain supporting evidence, uncertainty, and counter-signals.

Source viewers

Packet, binary, and artifact tools receive verified custody references and bounded coordinates.

Advisory research

Separate representability, executability, and derivability measurements assess the available evidence.

AttackMap and node inspector

Execution topology connects work to dependencies, context, receipts, and available actions.

WebRecon

Page/response evidence, source screenshots, review history, and endpoint gates share one map.

Workbench and continuation

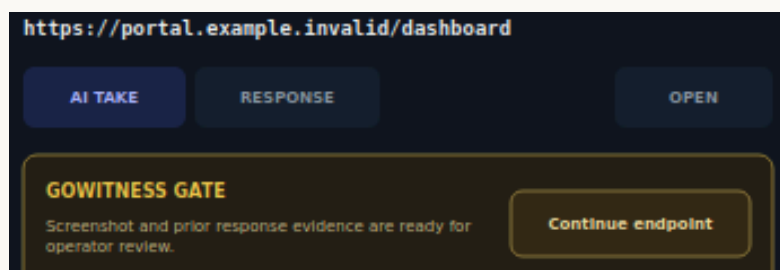
Searchable canonical records expose source navigation, context, reconciliation, and recovery.

TMG and SCG

The Trigger Methodology Graph proposes evidence-bound hypotheses; the Shadow Capability Graph diagnoses missing prerequisites.

Deterministic simulator

Controlled process and clock substitutions exercise production orchestration and persistence.



WebRecon endpoint review · Qt capture · synthetic fixture

Canonical engagement state

An engagement revision contains the related evidence, identity, authority, execution, and recovery records needed for a consistent decision. Validation checks that references resolve and that state transitions obey their declared rules before the revision is persisted.

Identity

Realm · principal · typed secrets

Authority

Scope · credentials · gates

Evidence

Custody · provenance · artifact roles

Execution

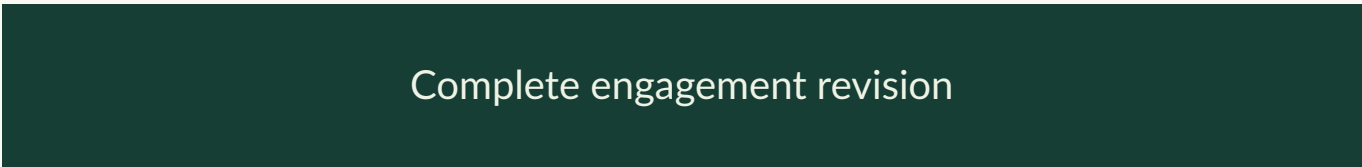
Actions · requests · receipts

Recovery

Resources · frontier · lineage

Revision

Reference closure · valid transitions



Related records are validated and persisted within one revision.

CONCEPTUAL DIAGRAM

Identity is explicit. A principal belongs to one realm, and embedded identity information must agree with its supplied context. Secret references declare their type and representation. Credentials retain provenance, applicability, and validity. Operations involving several artifacts require a digest-bound set whose semantic roles satisfy a versioned contract.

This complete boundary allows recovery to use a coherent historical state. Individual domains cannot be combined from mismatched revisions.

Atomic checkpoint persistence

The commit path validates the proposed engagement state, locks the engagement, and checks the expected head. It then builds an immutable generation, computes its hashes, inserts it, and advances the head within one PostgreSQL transaction.

01 Validate state

Reference closure and legal transitions

The state digest covers engagement content. A separate envelope digest binds the generation identity, parent, revision, event cursor, and timestamp. Compatibility records commit in the same transaction.

02 Check the head

Expected revision and parent generation

A repeated idempotency key returns the retained result only when its canonical payload is identical. A changed payload conflicts. Concurrent writers therefore receive an explicit stale-head or content conflict rather than silently replacing one another's work.

03 Build the generation

Content and envelope digests

04 Commit atomically

Generation, compatibility records, and head

Checkpoint publication occurs within one PostgreSQL transaction.

CONCEPTUAL DIAGRAM

Lifecycle and interruption

Typed events drive a deterministic lifecycle reducer. Completed, failed, and cancelled outcomes remain terminal. Cancellation records distinguish human decisions from branch, endpoint, policy, and shutdown causes.

CONDITION	RECORDED MEANING
Queued	Work exists; execution has not begun.
Running	A current owner holds the attempt and progress lease.
Interrupted	Nonterminal work has lost its runtime owner.
Unknown outcome	An effect may have occurred; observation or reconciliation is required.
Terminal	Completed, failed, or cancelled retains its final outcome.

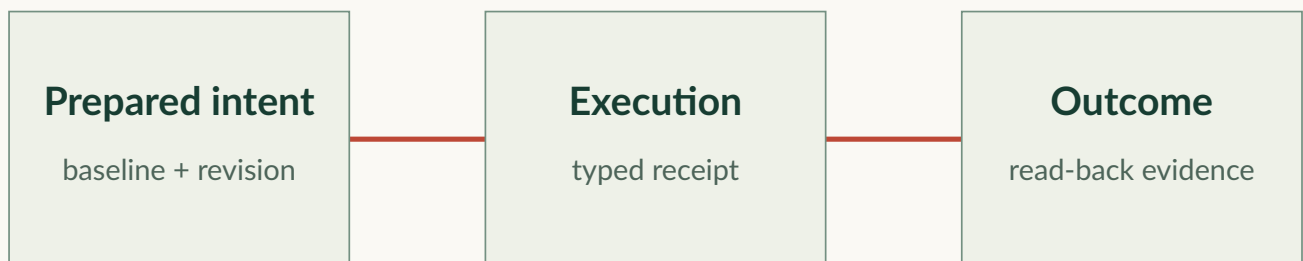
Lifecycle states preserve ownership loss and unresolved outcomes.
CONCEPTUAL DIAGRAM

A persisted nonterminal process with no runtime owner becomes interrupted. If an external effect may already have occurred, the action retains an unknown outcome until a read-back observation or reconciliation decision establishes the result.

The execution owner coordinates cancellation with completion, preventing competing writers from committing contradictory terminal states. Recovery retains the cause, attempt history, and ownership information needed to explain the transition.

Action receipts and outcome custody

The action coordinator checkpoints an intended change and its baseline before dispatch. A typed executor performs the bounded operation under a progress lease. A separate observation checks the resulting state before the coordinator commits the outcome.



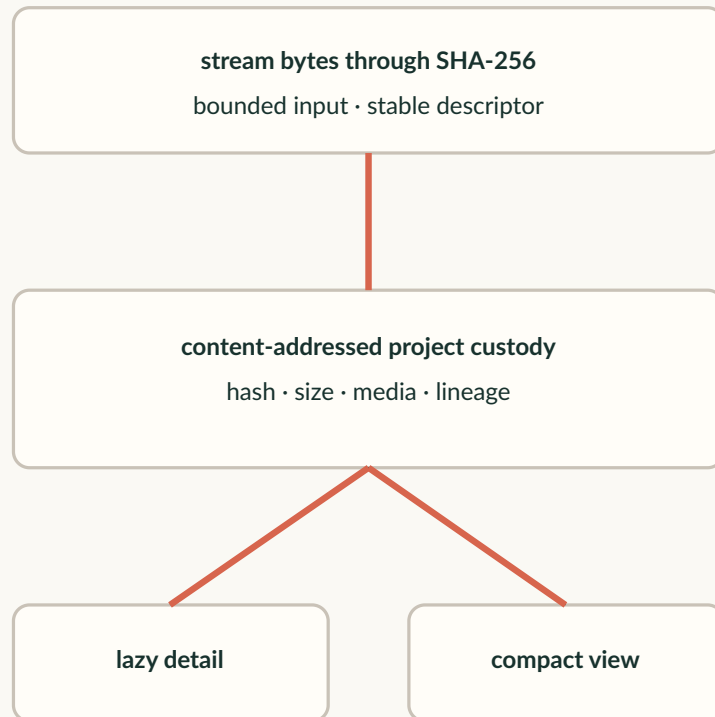
Intent, dispatch, and observed result retain separate records.
CONCEPTUAL DIAGRAM

The baseline distinguishes a new change from a condition that already existed. Execution receipts identify the dispatched work, while outcome evidence records the observed postcondition.

A lost worker or unverifiable result leaves the action unresolved. Recovery uses read-back or an attributed reconciliation decision to establish what happened before another effect can be authorized.

Content-addressed evidence

Raw specialist content is streamed through SHA-256 into project-scoped content-addressed storage. PostgreSQL records its digest, size, media metadata, and custody reference. Atomic linking and advisory locks deduplicate identical content within the project.



Raw custody serves multiple compact projections and source views.

CONCEPTUAL DIAGRAM

Operator and graph snapshots carry compact references. Detailed content loads when selected. Before a source viewer opens an artifact, the backend rechecks the retained object's integrity and supplies bounded coordinates or filters.

Shared objects require reference-aware reclamation. Deleting one run cannot safely remove raw evidence still referenced by another. A changed object or invalid source coordinate produces a visible navigation failure.

Execution-context timeline

An execution-context frame records the principal, host, process, isolation boundary, session, credential references, route, policy scope, and observation time. Each dimension is observed, unavailable, not applicable, stale, or conflicted. Unavailable values carry a reason.

CONTEXT DIMENSION	RETAINED FACTS
Identity	Principal, credentials, delegated authority
Placement	Host, process, isolation boundary, session
Reachability	Interface, route, tunnel, relay
Policy	Applicable scope and gate references
Validity	Observed, unavailable, not applicable, stale, or conflicted

Each factual dimension has its own observation status and time.

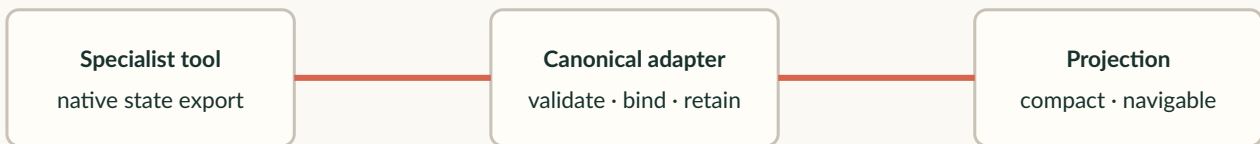
CONCEPTUAL DIAGRAM

Frames form a causal timeline. A successful local launch can add a child frame with an observed process ID. Changes to identity, namespace, credentials, route, session, or scope return a continuation to review.

AttackMap and Network Graph navigate through exact frame references. When a frame has no graph representation, the workspace reports the missing representation and retains the factual detail in the timeline.

Specialist interchange contracts

Specialist adapters preserve the information needed to revisit a result in its originating context. Packet selections retain filters and ranges; binary selections preserve address spaces and image bases; artifact selections retain bounded source coordinates. HTTP/browser and multi-environment records use the same versioned contract family.



Specialist output passes through a typed adapter into a compact projection.
CONCEPTUAL DIAGRAM

Each session and result carries project/run identity, source evidence, adapter and tool versions, revision lineage, applicability, and compatibility warnings. Raw content remains referenced separately from its compact projection.

The backend resolves canonical context for annotations and binds each note to an exact result revision. Unsupported fields and unresolved coordinates remain visible, allowing the operator to assess the available fidelity.

Source coordinates and viewer handoff

Source navigation begins with a canonical result or selection identity. Go verifies the retained artifact and resolves its coordinates before a viewer receives the handoff.

SOURCE FAMILY	BOUNDED NAVIGATION COORDINATES
Packet / protocol	Packet selection, filters, and ranges
Binary / debug	Address space, image base, and selected range
Artifact / mobile	Artifact identity and bounded source coordinates
HTTP / browser	Source evidence, request/response identity, and selection
Environment	Process, namespace, session, and context transition

A source-viewer handoff binds retained custody to the intended selection.
CONCEPTUAL DIAGRAM

Packet records preserve display filters, exact ranges, and the first selected packet. Reverse/debug records retain static and runtime address spaces, image bases, symbols, and offsets; address translation requires sufficient retained base information. Artifact/mobile records carry source and resource coordinates alongside the analyzed artifact's digest.

The handoff uses a private, same-user descriptor containing bounded values. Viewer capability remains explicit: stock JADX can open the artifact, while deeper coordinates remain available to the Workbench or a compatible plugin. The source tool's provenance survives use of a common viewer.

Deterministic evidence transformations

Deterministic analysis records connect datasets, transformations, experiments, and derived observations. Their relationships preserve which inputs were transformed or filtered and where results correlate, contradict, or converge.

01 Bound inputs

Exact source records and their revisions

Each ordered plan retains its hash, operation parameters, implementation version, and per-step input/result hashes. Baseline, result, and delta records retain rejected or unmatched inputs alongside validation and limitations.

02 Versioned transform

Deterministic operation and parameters

The backend accepts a bounded declarative operation catalog.

03 Derived result

Typed output with bounded custody

Investigation Graph renders the resulting records, while the Workbench exposes their derivation details. AI-proposed plans remain subject to the same typed backend validation.

04 Derivation history

Inputs, results, deltas, and retained lineage

Derived analysis preserves the relationship to its source records.

CONCEPTUAL DIAGRAM

Checkpointed semantic experiments

A semantic experiment binds a versioned recipe to its source result, execution context, scope, adapter version, and mutation preview. Missing inputs create a dependency wait. An unavailable adapter creates a specialist wait. Work requiring approval retains an exact gate identity and revision.

-
- | | | |
|-----------|---|---|
| 01 | Bind the plan
Recipe, source, context, scope, and preview | Each completed step writes a checkpoint before the next step begins. After interruption, recovery can retain the verified prefix and discard incomplete work. Changed inputs or a changed preview conflict before adapter dispatch. |
|-----------|---|---|
-
- | | | |
|-----------|--|--|
| 02 | Resolve prerequisites
Inputs, adapter availability, and exact approval | Protected values remain ephemeral request data. Adapter output passes through a hard byte limit into content-addressed storage, and cancellation retains its explicit cause through the execution owner. |
|-----------|--|--|
-
- | | | |
|-----------|---|--|
| 03 | Checkpoint each step
Retain every verified completed result | |
|-----------|---|--|
-
- | | | |
|-----------|--|--|
| 04 | Recover the prefix
Preserve completed work; reconcile interruption | |
|-----------|--|--|
-

Ordered experiment steps retain independently recoverable checkpoints.
 CONCEPTUAL DIAGRAM

Scheduler frontier and progress leases

The canonical scheduler evaluates verified capability evidence, current resource health, context validity, and deterministic decision receipts. It records the reason each work item is eligible, waiting, or unavailable. Priority and stable identity determine ordering among eligible items.

ELIGIBILITY INPUT	REQUIRED CONDITION
Capability	Verified, applicable evidence
Resource	Current health and valid dependency
Context	Exact identity, placement, route, and scope
Decision	Deterministically validated receipt
Lease	Owner and generation; heartbeat, progress, and overall deadlines

The frontier records eligibility and blockers against an expected revision.
CONCEPTUAL DIAGRAM

Every lease carries an owner and generation, an attempt, and separate heartbeat, progress, and overall deadlines. A worker may remain alive while useful progress has stopped; the progress deadline detects that condition independently.

Frontier updates require the expected engagement revision. Older chain paths still produce compatibility state during adoption, and complete retirement of fixed-order legacy scheduling remains unfinished.

Supervised AI decisions

AI work begins with a canonical evidence packet tied to an engagement revision and a recomputed context digest. Provider execution has bounded capacity, output, cache lifetime, and duration, together with a progress lease.

-
- | | | |
|-----------|--|---|
| 01 | Canonical evidence
Revision-bound packet and typed request | A registered deterministic validator produces a typed result and durable receipt. The receipt retains the evidence revision, provider/model, validator identity, response and result digests, validation errors, cache provenance, and time. |
| <hr/> | | |
| 02 | Provider work
Bounded reasoning under supervised ownership | The AI service cannot create or complete scheduler work directly. A stalled request requires a recovery transition. These contracts close the gap between displaying a waiting state and retaining a supervised, auditable decision request; adoption across older paths continues. |
| <hr/> | | |
| 03 | Deterministic validation
Identity, references, schema, and applicability | |
| <hr/> | | |
| 04 | Decision receipt
Durable result for policy and application services | |
-

A provider response becomes usable through deterministic validation.

CONCEPTUAL DIAGRAM

Isolated worker boundaries

An isolated worker receives one immutable execution envelope containing its run, stage, attempt, worker epoch, artifact generation, and input hash. It returns one bounded result envelope, while diagnostic output remains separate.

WORKER BOUNDARY BINDING	
Input	Immutable request envelope and input hash
Artifact	Exact admitted worker identity
Ownership	Provider generation and live resource lease
Output	Typed records and custody references
Recovery	Cancellation cause, cleanup, and retained result state

Worker inputs and results remain bound to admitted code and current ownership.
CONCEPTUAL DIAGRAM

A result can commit only while its artifact, generation, input hash, and stage still own the live lease. Proposed facts must belong to the artifact's declared output contract and pass coordinator-owned observation validation. Dependency waits must also be declared.

Registered isolated workers can be replaced while their coordinator remains active. Legacy in-process handlers require a scheduler boundary because they may already have changed controller memory. Their migration depends on explicit inputs and deterministic output validators.

Runtime composition kernel

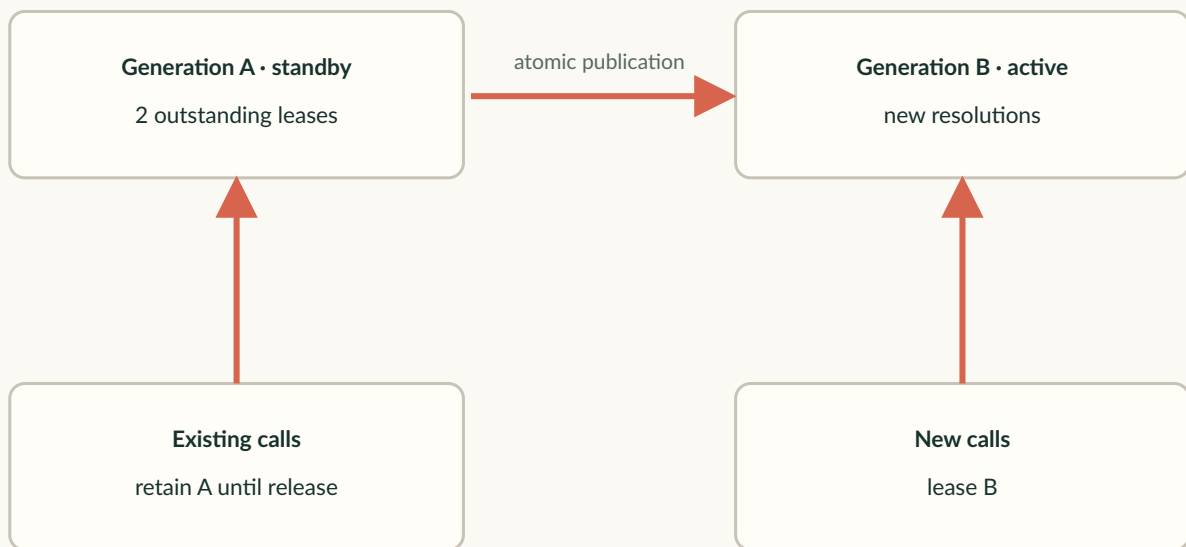
The runtime composition kernel manages typed providers, validated manifests, immutable generations, and consumer leases. A provider is prepared in an unpublished scope with pinned dependencies and recorded effects. Successful publication changes its availability atomically.

-
- | | |
|--|--|
| <p>01</p> <p>Define</p> <p>Typed service contract and dependencies</p> | <p>Consumers lease the exact generation they resolved. Replacement and cleanup respect those leases; setup failure unwinds effects already created. Cleanup executes once even when several callers wait for it.</p> |
|--|--|
-
- | | |
|---|---|
| <p>02</p> <p>Prepare</p> <p>Unpublished scope with recorded effects</p> | <p>Engagement state remains outside the replaceable provider. The kernel controls service availability and ownership, while application services retain evidence, policy, and durable workflow state.</p> |
|---|---|
-
- | | |
|--|--|
| <p>03</p> <p>Publish</p> <p>Atomic generation visibility</p> | |
|--|--|
-
- | | |
|--|--|
| <p>04</p> <p>Lease</p> <p>Exact generation retained by each consumer</p> | |
|--|--|
-
- | | |
|---|--|
| <p>05</p> <p>Drain</p> <p>Bounded reverse cleanup after release</p> | |
|---|--|
-

The kernel manages provider availability through one ownership model.
CONCEPTUAL DIAGRAM

Provider generations and replacement

A replacement provider must satisfy the required service contract, artifact identity, and dependency conditions before publication. New resolutions use the replacement generation. Existing calls retain their original generation until they release it.



Existing calls retain generation A while new resolutions use generation B.
CONCEPTUAL DIAGRAM

Commit drains the previous generation. Rollback restores its availability and drains the rejected replacement. Exclusive effects require controlled ownership so competing generations cannot acquire them unsafely.

These operations change a provider boundary. Previously completed external actions retain their own outcome history, and the replacement still requires health and semantic validation.

Process ownership and health

Managed resources retain separate desired state, process ownership, health, and applicability. A named, bounded observation establishes whether a resource is healthy enough for dependent work.

RESOURCE FACT	QUESTION ANSWERED
Desired state	Should the managed resource exist?
Ownership	Which supervisor and generation hold it?
Health	Has a named, bounded observation succeeded recently?
Applicability	Can this resource support the exact work and context?

Availability depends on four independently recorded resource facts.
CONCEPTUAL DIAGRAM

When a process disappears, its supervisor records interruption and reconciles the desired state. Backoff and crash-loop status remain explicit. Process-group cleanup uses bounded termination under the recorded owner.

Qt panes can detach from warm specialist windows while their dedicated supervisor retains the process. This distinction allows presentation changes without transferring lifecycle responsibility to an unrelated widget.

Autonomic Engineering Plane

The Autonomic Engineering Plane coordinates software repair above ordinary engagement execution. Its architecture covers incident observation, defect classification, a persistent repair session, validation, admission, component replacement, and verified return to the interrupted work.

01 Observe and classify
Retain the incident and select its repair owner

Product, orchestration, custody, and generic platform defects belong to that repair path. A missing target interaction may instead be routed to the separate Adaptive Capability Pipeline. The distinction preserves a clear owner for each kind of defect.

02 Repair session
Scoped source work with persistent incident history

An accepted repair session retains responsibility through completion. Implementation and validation of the complete loop remain in progress; reliable convergence across unfamiliar failures has not yet been established.

03 Validate and admit
Independent evidence and deterministic admission

04 Replace and hand back
Component swap and verified return to interrupted work

AEP retains ownership from the accepted incident through verified hand-back.

CONCEPTUAL DIAGRAM

Adaptive Capability Pipeline

The Adaptive Capability Pipeline retains one persistent generation session for a scoped capability contract. Phase records bind their inputs and artifacts by digest. Validated receipts allow compatible completed work to be reused, while live effect validation remains mandatory.



A generation session retains candidate artifacts and their admission evidence.

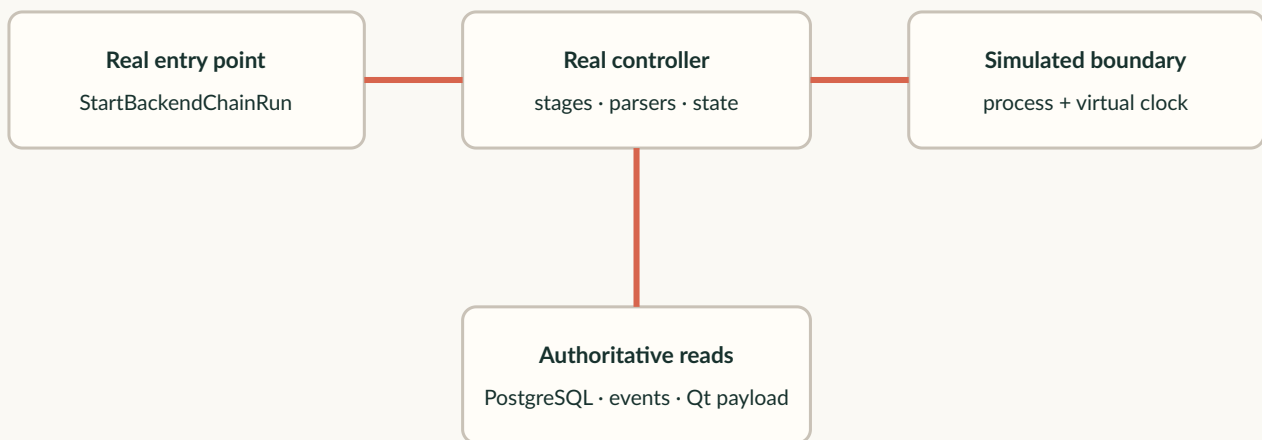
CONCEPTUAL DIAGRAM

A candidate becomes visible only through a manifest binding its source, metadata, and optional tests. Rejection receipts remain immutable regression inputs across continuation and controller replacement.

Contamination review uses a separate workspace with read-only candidate access and bounded provenance. Its verdict informs deterministic admission. Generation-session persistence does not preserve external process ownership; interrupted effects require their own reconciliation.

Production-orchestration simulation

The deterministic simulator invokes the real chain entry point and allows production orchestration to choose branches. For one run, it substitutes process-launch and clock behavior. Synthetic byte chunks then pass through the same stream handling and parsers.



Production control and authoritative reads surround a simulated process boundary.

CONCEPTUAL DIAGRAM

Assertions examine PostgreSQL, immutable events, canonical snapshots, and operator-facing projections. Unmatched calls, undeclared activity, incomplete dependencies, and duplicate effects fail the scenario.

This establishes bounded evidence for orchestration, persistence, and recovery mechanics. Real tool installation, operating-system behavior, browser processes, and network fidelity require separate environmental validation.

CI and cache custody

The Go CI routes use isolated checkouts and private scratch while reusing admitted build and module caches. Cache-owner checks verify ownership, ancestry, and the cached toolchain before the route executes.

RUN 1599 GATE	DOCUMENTED RESULT
Cache custody	Passed
Overlay	Passed
PostgreSQL	Passed
Root contracts	Three failures
Process cleanup	Ambient CLI process tree retained

Latest documented pushed Go receipt; subsequent repairs await a new result.
DOCUMENTED CI STATUS

Ultra records bounded receipts for overlay, compile, baseline, race, repetition, fuzz, fault, and artifact lanes. Receipt validation checks the expected inventory, file identity, containment, hashes, and durable staging. Skipped or unavailable lanes retain distinct statuses.

The latest documented pushed run, 1599, passed cache, overlay, and PostgreSQL gates before failing three root contracts and retaining an ambient CLI process tree. Later repairs await a new pushed result. Operator visual coverage is selected by changed-component scope, and a newer passing result remains unconfirmed.

Verified artifact deployment

Deployment verifies a complete staged artifact set before publication. Manifests bind each component's digest, source revision, and ABI expectations. A final readiness marker records successful verification.

01 Stage the set

Complete component files in a private incoming set

The operator and backend undergo paired ABI checks before replacement. Publication is atomic, and rollback distinguishes a complete restore from an incomplete one. The watchdog retains ownership of its embedded operator; specialist processes remain under their dedicated supervisors.

02 Verify

Manifest, source revision, digest, and readiness

Historical reports cover tamper rejection, rollback contracts, process-group cleanup, and a display-host replacement exercise. Real Chromium descendant behavior and wider display-host acceptance remain under validation.

03 Preflight

Paired Go and Qt ABI compatibility

04 Publish or restore

Atomic replacement with explicit rollback status

Artifact integrity and compatibility are checked before publication.

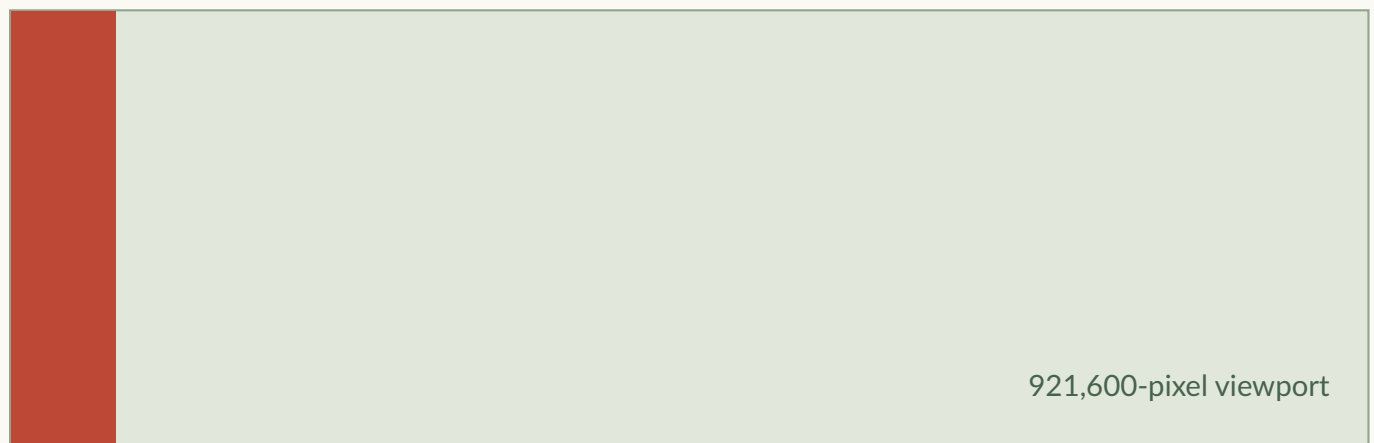
CONCEPTUAL DIAGRAM

Rendering and runtime observability

Native graph surfaces preserve stable geometry and restrict repainting to affected regions. AttackMap records paint count, duration, dirty pixels, and visible graph elements. WebRecon uses viewport culling, and Network Graph caches its background grid.

71,462

dirty pixels



Dirty-region update in the retained operator fixture: 71,462 of 921,600 pixels.

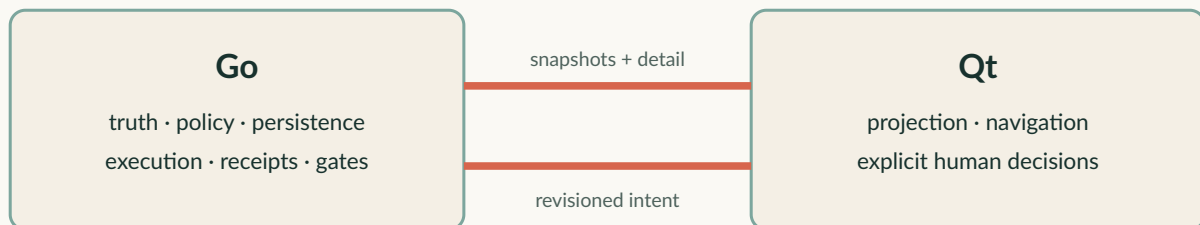
HISTORICAL MEASUREMENT · JULY 2026

The Go bridge polls away from the UI thread and yields between bounded backlog pages so input and painting can proceed. A reset requests a fresh snapshot.

Historical operator verification recorded a 71,462-pixel dirty update within a 921,600-pixel viewport, together with settled-idle assertions. Those measurements describe the retained fixture and build. Runtime leases, progress deadlines, events, and provider generations supply corresponding backend diagnostics.

Go/Qt component interfaces

The Go backend exposes versioned JSON payloads through a native C ABI. Qt receives snapshots and deltas, loads details on demand, and submits commands carrying record identity and expected revisions. Go resolves the canonical facts behind those commands.



Versioned state flows to Qt; attributed intent returns to Go.
CONCEPTUAL DIAGRAM

Required-symbol changes advance the numeric ABI. Existing exports remain additive where compatibility is supported. Process bundles declare their launch, writable-state, output, and ownership contracts.

Specialist source viewers receive private, same-user descriptors bound to artifact digests and bounded coordinates. These interfaces allow components to evolve while retaining explicit compatibility and ownership checks.

Technical capabilities

The shared contracts support work across web evidence, packet/protocol records, reverse/debug state, analyzed artifacts, and environment transitions. Each family can retain its native coordinates while participating in the same evidence, context, and recovery model.

UNDERLYING CONTRACT	TECHNICAL CAPABILITY
Canonical state	Consistent recovery and evidence inspection
Typed custody	Fidelity-preserving source navigation
Revised decisions	Scoped review and attributed continuation
Generation leases	Component replacement with retained ownership
TMG / SCG	Evidence-bound hypotheses and prerequisite diagnostics
Native projections	Responsive graph inspection and bounded detail

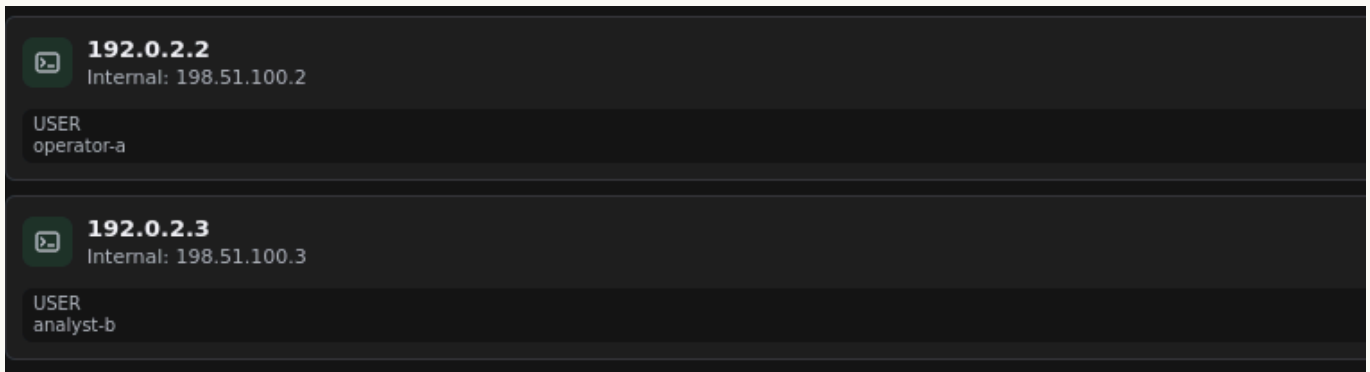
Operator capabilities derive from shared state and interface contracts.
CONCEPTUAL DIAGRAM

The operator can follow a conclusion to its source, inspect the authority behind an action, and distinguish recoverable interruption from missing prerequisites. Deterministic summaries provide an evidence index for findings and report assembly. TMG and SCG add diagnostic hypotheses and prerequisite analysis.

Breadth varies by family. Typed interchange and fixture coverage are implemented more widely than representative live validation. The plugin registry keeps rare or manual-required work visible within the same ownership model.

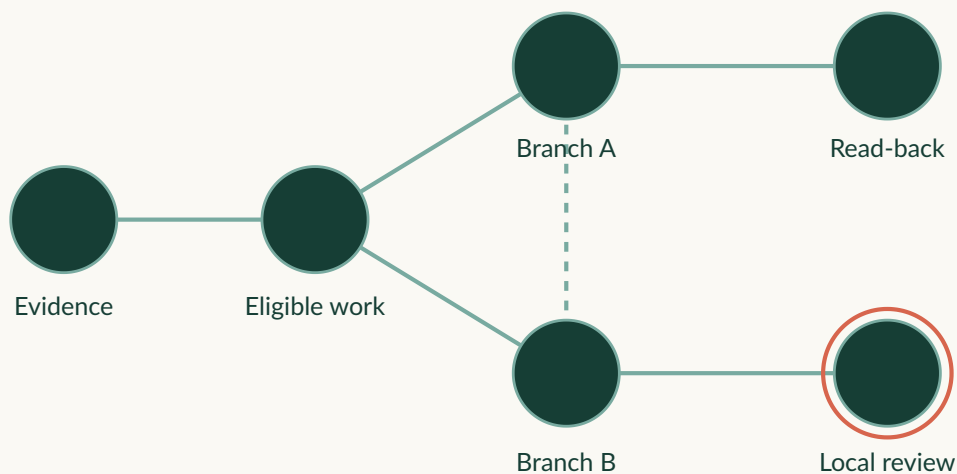
Native operator and AttackMap

The operator uses QML for menus, project navigation, overlays, and workspace controls. Native QWidget/QPainter surfaces render analytical content. Sessions and external-tool history remain accessible within the same window.



Sessions workspace with synthetic project data.

QT CAPTURE · SYNTHETIC FIXTURE



Dependencies, concurrent work, and a local review gate.

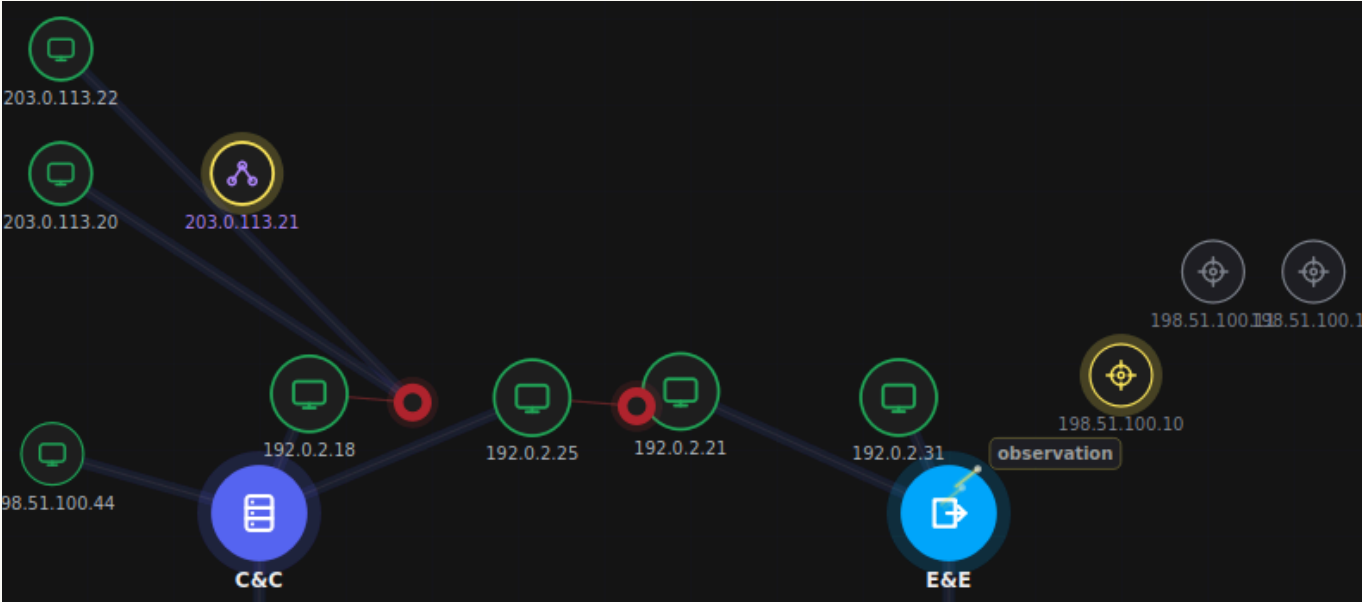
CONCEPTUAL DIAGRAM

AttackMap presents plan nodes, runtime instances, dependencies, endpoint lanes, and review state. Go owns their semantics. Stable identities preserve graph placement as new work arrives, and runtime deltas repaint the affected region.

The node inspector requests canonical detail on demand. It exposes blockers, evidence, exact context, active tools, and available actions with their expected consequences. Detailed streams remain outside the steady-state graph snapshot.

Network and context navigation

Network Graph renders hosts, sessions, relays, tunnels, targets, and packet relationships on a native canvas. Pan, zoom, drag, fit, and stable focus support inspection of the current topology.



Network Graph with synthetic host and session data.
QT CAPTURE · SYNTHETIC FIXTURE

Identity

Exact graph node

Context

Observed frame reference

Absence

Unresolved representation

Context navigation matches observed frame references against exact node identities or labels. Unavailable route, namespace, or host relationships remain unresolved. The factual context is still available through the timeline.

The retained standalone image uses synthetic topology data. Live topology ingestion and environmental behavior require their own verification.

WebRecon and endpoint review

WebRecon organizes page and response evidence into stable cards with directional relationships. Headers, technologies, screenshots, and review history enrich existing evidence identities. Text updates preserve the camera and geometry; topology changes trigger layout.



Web evidence linked by navigation and response relationships.

QT CAPTURE · SYNTHETIC FIXTURE

GOWITNESS GATE

Screenshot and prior response evidence are ready for operator review.

Continue endpoint

AI TAKE

Authenticated dashboard. Multiple XHR endpoints expose user and organization records; verify object-level authorization across tenant boundaries.

Selected evidence and a pending endpoint review.

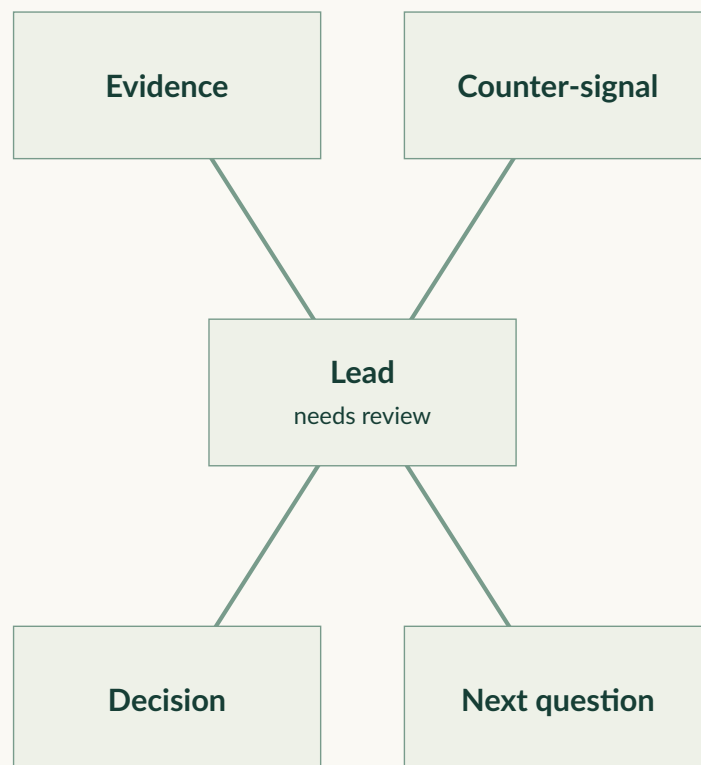
QT CAPTURE · SYNTHETIC FIXTURE

Browser review creates a persisted gate for one endpoint. Independent endpoints retain their own workers and can continue finalizing evidence. Continue decisions bind the exact gate identity and revision, remain idempotent across restart, and produce attributed audit events.

The selected-evidence pane displays the source record and review control. Human-only challenges use explicit response or browser-handoff boundaries. The native renderer culls offscreen content and stops repainting once settled.

Investigation Graph

Investigation Graph presents hypotheses and conclusions as readable cards, with supporting observations, uncertainty, and counter-evidence. Routine artifacts and observations collapse into bounded evidence groups while the complete machine graph remains available underneath.



A conclusion linked to supporting observations and counter-evidence.

CONCEPTUAL DIAGRAM

Selecting a conclusion reveals its support. Inspect opens canonical identifiers, revisions, hashes, custody, context, and model-input provenance in the Workbench. AI assessments retain their own status and cannot change deterministic facts or execution state.

Attention is organized within each evidence cluster. Meaningful changes in priority bands or membership trigger bounded settling; stable content remains still.

Workbench and continuation

The Investigation Workbench combines evidence, specialist records, execution-context timelines, and continuation state in searchable rows. Selecting a record loads its detailed session once and preserves the stable identity of each child.

WORKBENCH RECORD INSPECTION AND CONTINUATION

Canonical detail	Stable identity, revision, evidence, and custody
Source navigation	Packet, binary, artifact, and environment coordinates
Context timeline	Identity, placement, route, scope, and transitions
Pending reconciliation	Separate attributed acceptance or rejection
Accepted intent	Exact command identity retained across interruption

Canonical record detail, source navigation, and pending reconciliation.
CONCEPTUAL DIAGRAM

Continuation distinguishes active, interrupted, imported, conflicted, and resumable work. Each action identifies the checkpoint, context, ownership revision, and gate it requires. Pending reconciliation requires a separate attributed acceptance or rejection before lifecycle state changes.

Packet filters and ranges, binary address spaces, artifact coordinates, and environment transitions remain available through source navigation. Protected bindings are ephemeral. Immutable intents and receipts preserve the identity of an accepted command across a crash.

Evaluation methodology

Validation progresses from contract review through parser or stage fixtures, production-orchestration simulation, representative live labs, and broader environmental acceptance. Each level introduces dependencies that the earlier levels leave controlled.

01 Contract review
Declared behavior and proposed methodology

02 Parser or stage fixture
Supplied inputs and local transitions

03 Orchestration simulation
Production state and coordination under controlled failure

04 Representative live lab
Real tools, processes, services, and operating systems

05 Broader acceptance
Unusual configurations and sustained integration

Research evaluation separates representability, executability, and derivability. A procedure may fit the typed vocabulary while lacking either a usable executor or sufficient source evidence. Frozen registries, identity-disjoint cohorts, and retained pre-change results preserve the meaning of held-out comparisons.

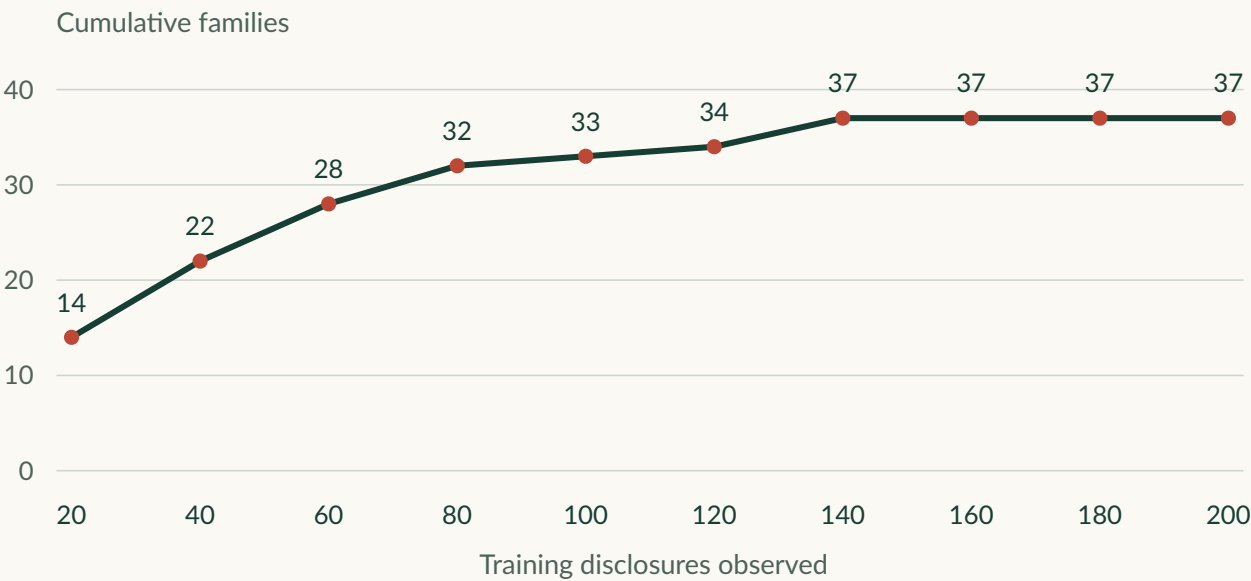
Counterfactual fixtures vary identities, ordering, absent evidence, and competing candidates. Historical CI, synthetic simulation, research measurements, and reported field outcomes retain their own evidence scope. Current software or environmental readiness requires a result tied to the relevant revision and conditions.

Each evaluation level introduces dependencies absent from the previous level.

CONCEPTUAL DIAGRAM

Research measurements

The disclosure study used 200 identity-disjoint training records and 40 held-out records balanced across four severity bands. Training represented 37 recurring families. Three renamed synthetic rotations per candidate produced 600/600 clean activations and 200/200 shadow-calibrated candidates.



Training disclosures and cumulative represented families.
VERSIONED STUDY · 200 TRAINING RECORDS

Held-out family coverage was 39/40. The cumulative family count reached 37 at disclosure 140 and remained there through 200. The cohort enforced identity separation; strict chronological separation was constrained by the available severity-balanced records. Live-exercised and admitted candidate counts remained zero.

A separate advisory study reported 50/50 discovery records and 24/24 held-out records represented without analyzer errors or missing primitive classes. Executability and derivability remained false where evidence was insufficient. Twenty-six held-out records remain untouched.

Retained verification and autonomous outcomes

Historical backend and operator verification records cover transactional PostgreSQL state, lifecycle reconciliation, specialist interchange, native rendering, and artifact integrity. The retained operator record includes software-rendered screenshots, graph-density fixtures, and paint telemetry.

RETAINED EVIDENCE	SCOPE
Backend / operator	Persistence, lifecycle, specialist contracts, rendering, integrity
18/18 runs	Three production discovery scenarios
40/40 runs	Reusable synthetic mechanics
Reported field outcomes	Autonomous completions; exact outcome logs unavailable

The retained records distinguish engineering checks, simulation, and field reports.
EVIDENCE CLASSES

A simulator report records 18/18 production-orchestration runs across three discovery scenarios and 40/40 reusable-mechanics runs. Later documentation records eleven authored mechanics scenarios plus three PostgreSQL-backed production cases. Those inventories retain different scopes; only the discovery cases traverse their matching production handlers.

Reported autonomous outcomes include P.O.O, a hard Windows CTF, an insane Windows HTB lab, and approximately two easy Linux/web HTB labs, completed without manual intervention. Exact logs for those outcomes were not located in the retained record.

Current gaps

Canonical state, typed adapters, and modern approval contracts coexist with older execution paths. Complete scheduler and AI adoption remains unfinished. The historical chain assessment records high-impact stages that can follow a single launch approval without individual stage gates.

BOUNDARY	REMAINING LIMITATION
Legacy adoption	Canonical scheduler and AI integration remains incomplete
Authority	Some legacy stages follow a single launch approval
Adapter breadth	Authentic exports and representative environments remain necessary
Recovery	Projection outages, browser descendants, and unusual environments
AEP / research	Unfamiliar-failure convergence and wider generalization remain unproven

Current limitations retain distinct implementation and evaluation scopes.
CONCEPTUAL DIAGRAM

Broader adapter families require authentic external exports and representative live validation. Archived-project migration, external Neo4j outage recovery, browser descendants, unusual virtual machines, and hardware or kernel behavior remain open environmental work.

AEP implementation is still stabilizing. Reliable repair and return-to-work across unfamiliar failures has not yet been established. TMG and SCG remain diagnostic research, with no admitted methodology claimed from the disclosure corpus.

Development priorities

Near-term work centers on canonical scheduler and AI adoption, representative adapter validation, and fresh passing CI results for repaired contracts. Recovery checks need to cover process loss, stale ownership, projection outages, and archived state under repeatable environmental conditions.

-
- | | | |
|-----------|--|--|
| 01 | Canonical adoption
Scheduler, supervised AI, and repaired CI contracts | <p>AEP evaluation must retain the complete incident history through diagnosis, repair, independent validation, admission, replacement, and verified hand-back. Field outcomes need exact run, revision, environment, and terminal evidence where those records can be recovered or collected.</p> <p>Research development depends on unrelated evidence and unchanged held-out cohorts. New capability claims require counterfactual coverage and explicit limits before broader acceptance.</p> |
| 02 | Environmental verification
Ownership loss, outages, archived state, and adapter fidelity | |
| 03 | Repair-loop evaluation
Incident history, independent validation, and hand-back | |
| 04 | Research generalization
Unrelated evidence and unchanged held-out cohorts | |
-

Development priorities are tied to specific unresolved evidence requirements.

CONCEPTUAL DIAGRAM

References and technical review

The technical record comprises the engagement architecture, runtime composition and lifecycle contracts, specialist interchange, native operator contracts, historical verification reports, simulator records, and versioned research studies.

A01	Authoritative Engagement Architecture	A05	Execution Graph Lifecycle
A07	Canonical Specialist Contracts	A08	Specialist Adapters
A10	Investigation Attention	A11	Objective Review Contracts
A12	Specialist Plugin Registry	A13	Host Execution Attribution
C02	Archive Capture Integrity	D01	Deployment and Process Ownership
G01	Component Architecture	G03	Native FFI/API Surfaces
G04	Reporting Workflow	G07	Historical Chain Assessment
R01	Runtime Composition Kernel	R02	Runtime Lifecycle Adoption
R03	ACP Vertical Session	R04	Autonomic Engineering Plane Contract
R05	Isolated Stage Workers	M01	Trigger Methodology Graph
M02	Disclosure Generalization Study	M03	Shadow Capability Graph
M04	Advisory Abstraction Research	Q01	Qt Operator Architecture
Q02	AttackMap V2 Contract	Q03	WebRecon Native Parity
Q04	Investigation Graph and Workbench	Q05	CAPTCHA Human Gate Contract
E03	691 Verification Record	E04	CI Evidence and Cache Custody
E05	Operator Frontend Evidence	V01	Chain Simulator and Validation Ladder
V02	Simulator Architecture	V03	Simulator Execution Report
V04	Simulator Live-Escape Audit		

A private technical review would follow a synthetic engagement revision through evidence inspection, a review decision, interruption, and recovery. The same session could examine the associated contract and verification record.

REDC2 is independently developed technology with ongoing implementation and environmental validation.